Clase 2

@September 2, 2026

Introducción

El objetivo de esta sesión fue lograr que nuestro ecosistema de proyectos se comunique correctamente. Recordemos la regla de oro arquitectónica: **el Engine (motor) no compila como un ejecutable, el Sandbox sí.** Para acoplar las características de C++ al Engine, le agregamos un archivo `source.cpp` (aunque por sí solo no compilará un programa final, le dice al sistema que es un proyecto de C++).

Desarrollo

Sistema de Dependencias DirectXTK

Añadimos un nuevo proyecto a nuestra solución llamado **DirectXTK 2022**. Para que todo funcione, configuramos las referencias en cadena:

- El **Engine** depende de **DirectXTK**.
- El **Sandbox** depende del **Engine**.

Configuración de la API y Exportación de la DLL

Para que el Sandbox pueda usar las herramientas del Engine, necesitamos exportarlas. Creamos un código de "API" (Un archivo de cabecera `.h`) dentro del Engine.
Este archivo sirve para decirle al compilador que el Engine va a funcionar como una DLL y exporta sus recursos para que el Sandbox pueda consultarlos y ejecutarlos.

> *En C++, una DLL puede funcionar perfectamente sin tener que definir manualmente una función `DLLMain`.*

Inicialización en Engine.cpp

Dentro de `Engine.cpp`, creamos un puntero hacia nuestra DLL. Este recurso será el encargado de hacer el trabajo pesado inicial:

- Crear el "Device" (la conexión con la tarjeta gráfica).
- Recibir la información de nuestro equipo.
- Compilar y enlazar DirectXTK.

Prueba mínima de Éxito

Configuramos correctamente las rutas de salida de nuestros binarios (`.exe`, `.dll`), librerías (`.lib`) y archivos intermedios. Realizamos una prueba mínima para confirmar que la arquitectura está sana. Sabemos que tuvimos éxito porque logramos:

1. Compilar la DLL del Engine.
2. Generar el archivo `Engine.lib`.
3. Enlazar todo desde el Sandbox.
4. Inicializar DirectXTK.

Próximos pasos

En la siguiente sesión dibujaremos nuestro primer triángulo. Posteriormente trabajaremos con objetos tridimensionales y, una vez que entendamos el proceso, vamos a abstraer toda la clase.

Conclusión

Esta clase la verdad me enojó mucho por la forma en la que el profe explicó en algunas partes. Me enojé tanto que me fui a dormir.

A veces parece que las cosas están siendo leidas desde una traducción del ingles a español, con sus estructuras semánticas diferentes a las nuestras. Cosas como el uso de voz pasiva son entendibles y totalmente normales en el inglés pero no mucho en español. Y me parece que las materias escolares merecen un trato muy especial referente al uso del lenguaje para ser comunicadas.

Porque estas materias ya son muy pesadas cognitivamente. Al no comunicarlas de forma digerible, estamos imponiendo un doble trabajo mental a los alumnos.

Uno necesario de afrontar, ya que los temas son inherentemente complejos, y otro totalmente evitable que es descifrar las palabras que te acaban de decir.

La verdad es que el profe no está mal por hablar como sea que habla, yo también uso la voz pasiva y estructuras semánticas que no son comunes de vez en cuando. Realmente mi problema era el chapuzón que tenía que darme a este mundo de las gráficas computacionales y me tenía que quejar de todo porque si es un chapuzón doloroso.

Glosario

DirectXTK (DirectX Tool Kit)

Es una colección de clases y herramientas de ayuda (helpers) escritas en C++ por Microsoft. Nos ahorra tener que escribir desde cero matemáticas de vectores, carga de texturas, fuentes de texto y modelos 3D, facilitando mucho el trabajo con DirectX 11.

Lógica Aplicada

Se refiere a tomar conceptos abstractos o teóricos (como matemáticas o algoritmos) y traducirlos en código concreto que resuelva un problema específico dentro del motor gráfico (por ejemplo, cómo estructurar la memoria para que el Sandbox y la DLL se comuniquen sin fallar).

API (En el contexto de la DLL)

En esta clase, cuando hablamos de "un código llamado API", nos referimos a unas macros (instrucciones como `__declspec(dllexport)` y `__declspec(dllimport)`). Sirven como un "puente" o "etiqueta" que le dice al compilador: *"Oye, esta función específica del Engine la voy a prestar para que la use el Sandbox"*.

Abstracción

En programación, significa ocultar los detalles complejos y repetitivos detrás de una interfaz o función sencilla. Por ejemplo, en lugar de escribir 50 líneas de código cada vez que queremos dibujar un triángulo, "abstraemos" ese código creando una función llamada `DibujarTriangulo()`. Así, el código principal (el Sandbox) queda limpio y fácil de leer.